

ARC-AGI-3 Part 1:形式化与方法综述(第二版)

Bo Hou · 2026-10-04 · 系列:ARC-AGI-3 世界模型推导 · 第二版

本篇是系列第一部分:把问题形式化、统一符号,再用同一套符号重写现有方法,找出空白。第二版经过逐节审阅,第 11 节记录了审阅中的关键修正。

0 大白话:这道题在考什么

ARC-AGI-3 是一组没有说明书的小游戏:画面是 64×64 的彩色格子,agent 只能按按键、点格子,规则和目标都要自己摸索。

- **看到什么:**每一步一张 64×64 的画面,每格是 16 种颜色之一;能量条、提示框也画在里面。另外知道当前第几关、游戏结束没有、哪些动作可用。
- **能做什么:**最多 7 个动作加重开。上下左右、一个”交互”键、点击某个格子、撤销;但同一个键在不同游戏里意思不同,每个游戏只开放其中一部分。
- **要做什么:**每个游戏有好几关,后面的关更难、权重更高。过关条件不告诉你,要自己试出来。
- **怎么算分:**和第一次玩的普通人比动作数。用的动作是人的 2 倍,这关只拿 1/4 分;没过的关是 0 分。所以乱试很亏,探索也要花钱。
- **评测条件:**Kaggle 单卡、断网、每次提交最多 9 小时;公开 25 个游戏可以本地玩,评测用另外 110 个私有游戏,机制有意设计成和公开集不同。

一句话:它考的不是会不会玩某个游戏,而是能不能用很少的尝试学会一个没见过的游戏。

1 问题形式化

每个游戏是一台未知的确定性 Moore 机:内部状态 z 决定画面 x ,agent 只看得到 x 。观测字段已对照 arc-agi 0.9.9 / arcengine 源码核实。

1.1 符号表

符号	含义
$g \in G$	一个游戏; G_{pub} 为 25 个公开游戏, G_{priv} 为 110 个私有游戏
$\ell = 1, \dots, L_g$	关卡序号, L_g 为关卡数(win_levels)
t	真实动作步数
$x_t \in \{0, \dots, 15\}^{64 \times 64}$	第 t 步后的帧(一步可能返回多帧,取最后一帧)
σ_t	游戏状态:NOT_FINISHED / WIN / GAME_OVER
c_t	已通过的关卡数
$A_t \subseteq \{0, \dots, 7\}$	可用动作:0 RESET,1-4 方向,5 交互,6 点击,7 撤销
a_t	动作; $a_t = 6$ 时带坐标 $(u, v) \in [0, 63]^2$
$o_t = (x_t, \sigma_t, c_t, L_g, A_t)$	完整观测
z_t	真实内部状态

符号	含义
T^*, R^*	真实转移与渲染
$D_t = \{(x_i, a_i, x_{i+1})\}_{i < t}$	交互记录
ϕ, s	物体抽象映射与物体级状态(1.3 节)
$h = (Enc_h, T_h, R_h, G_h)$	一个 world model 假设(3.2 节)
$\Lambda = (\theta, harness)$	外层先验:模型权重 + 提示词/工具/流程
$b_{g,\ell}, n_{g,\ell}$	关卡 ℓ 的人类基线动作数、AI 动作数

1.2 动力学:z、T*、R*

$$z_{t+1} = T^*(z_t, a_t), \quad x_t = R^*(z_t), \quad a_t \in A_t$$

- **z**:游戏类此刻全部成员变量的取值(物体位置、能量值、开关、关卡、计时器……)。
- **R***:把 z 画成帧。画出来的部分(能量条、物体)可见;没画的部分(运动方向、冷却计时、未显示的开关)是隐藏状态。类比:z 是类的私有成员,x 是它的外在表现。
- **T***:确定性规则。输出只由状态决定(Moore 机),帧是 z 的投影而不是 z 本身;Tycho 称之为参数化、渲染式的确定性 Moore 机。
- 同一 (x, a) 若观察到多个 x' ,说明存在画面外状态,需要用历史推断。
- 先验:只依赖 Core Knowledge(物体性、几何与拓扑、能动性;二手资料另列基础物理),不用数字字母;数量以长短、个数等视觉形式出现。

1.3 物体与物体状态

物体级状态 s 是一棵树:物体可以包含子物体;每个物体 $o = (\kappa, \tau, v)$ 。

- **κ 身份**:跨帧追踪同一个物体。
- **τ 类型**:由颜色与形状掩码决定,与位置无关。
- **v 属性**:位置 = 包含物体像素集 P 的最小轴对齐矩形左上角 $(\min r, \min c)$,尺寸 $h \times w$;形状 = 外接框内二值掩码 $M \in \{0, 1\}^{h \times w}$;外观 = 颜色图 $C \in \{0, \dots, 15\}^{h \times w}$ (非本物体格标透明);层次 d (前后遮挡)。(位置, M, C) 可以把物体原样画回帧上。
- **遮挡**: P 取物体完整范围(被遮部分按跨帧追踪补全),否则位置会跳;所以 $s_t = \phi_h(x_{t-k}, \dots, x_t)$ 要用历史,对应 Core Knowledge 的物体恒存性。
- **ϕ_o 与 ϕ_h** :同色 4 邻接连通块分割 ϕ_o 是默认先验;实际使用的 ϕ_h 属于假设 h ,可被数据修正(如把总一起动的多个块合并)。
- **作用不在 ϕ 里**:“我控制的、会补能量的、是终点”这类语义只能通过交互得知,属于 T_h 与 G_h 。

2 计分与全局目标

目标是用尽量少的真实动作通过尽量多、尽量靠后的关卡;超出人类基线的动作按平方衰减。单关得分(未通关为 0):

$$r_{g,\ell} = \min(1.15, (\frac{b_{g,\ell}}{n_{g,\ell}})^2)$$

单局得分(按关卡序号加权)与总分:

$$S_g = \frac{\sum_{\ell=1}^{L_g} \ell r_{g,\ell}}{\sum_{\ell=1}^{L_g} \ell}, \quad S = \frac{1}{|G|} \sum_{g \in G} S_g$$

- $b_{g,\ell}$ 是首次接触玩家动作数的 upper median。
- $n_{g,\ell}$ 怎么计(arc-agi 0.9.9 scorecard.py):ACTION1-7 每次 +1;普通 RESET 同时计 1 次 reset 和 1 个动作;完全重置(full_reset)新开一条游玩记录,动作数从 0 重新计。源码单关上限为 115(百分制),与 1.15 一致。Kaggle 服务端是否同一代码未能直接核实。
- 1.15 的上限允许比人类少约 7% 的动作仍封顶($\sqrt{1.15} \approx 1.07$)。
- 例:人用 22 步,AI 用 44 步得 0.25;平均分 1.6% 不能反推成”每关多用 8 倍”,因为没过的关卡 0 计入。

全局目标:策略 π (含建模与规划)最大化私有集期望总分:

$$\pi^* = \arg \max_{\pi} E_{g \sim G_{priv}} [S_g(\pi)]$$

3 两层建模框架

外层定义”用什么语言写游戏程序”,内层在这门语言里现场写出当前游戏的程序并用它过关;两层合起来是 meta-learning。

3.1 Λ 、 $\mathcal{H}$ 、 h :用”基础类”来理解

- Λ (外层):一套基础类和交互原语的库,如”可移动物体””墙””计数条”,”碰撞触发效果””满足条件才放行”。实际系统里 $\Lambda = (\theta, \text{harness})$: θ 是 LLM 权重,harness 是提示词、工具和流程。
- $\mathcal{H}$ (假设空间):用 Λ 能写出的所有游戏程序。 Λ 越丰富, $\mathcal{H}$ 越大,能表达的游戏越多,搜索越难。
- h (内层): $\mathcal{H}$ 里一个具体程序:有哪些物体、各属哪个类,动作作用在谁身上,碰到一起发生什么,怎样算赢。
- 外层学习的显式目标:学会”快速、简洁地生成一个能生成这个游戏的程序”。学的不是新规则,而是”学出新规则”的能力;私有集机制有意分布外,所以 Λ 应存底层原语而不是具体规则。

3.2 内层:推断隐状态、转移、渲染

T^* 不可观测,因为 z 是隐变量; h 自己维护对 z 的估计 $\hat{z}$,用历史推断,再渲染回帧和观测比较:

$$\hat{z}_t = \text{Enc}_h(x_{\leq t}, a_{< t}), \quad \hat{z}_{t+1} = T_h(\hat{z}_t, a_t), \quad \hat{x}_{t+1} = R_h(\hat{z}_{t+1})$$

- 只在所有状态都画出来时才有 $\hat{z} = x$;”推断器 + 转移 + 渲染”与变分 world model 的”编码器 + 转移 + 解码器”对应,但这里三者是程序。
- T_h 在可观测意义上逼近 T^* ,且只需在对过关有用的规则上逼近。

每一步的循环(动作永远在真实环境执行):

1. 观察 x_t ,把 (x_{t-1}, a_{t-1}, x_t) 加入 D_t 。
2. 校验并修改 h ,使其与 D_t 一致。
3. h 足够可信 $\rightarrow$ 在 T_h 里搜索通往 G_h 的序列;否则 $\rightarrow$ 选让候选假设预测分歧最大的动作。
4. 执行中比对预测与真实帧,不一致回到第 2 步。

可信判据:与 D_t 一致且 $C(h') \leq C(h) + \varepsilon$ 的所有候选 h' ,在计划路径上的预测全部相同。不可信时分歧最大的一步,恰好就是第 3 步该探索的动作。 ε 待定,是否完全闭合需边做边看。

3.3 建模 cost

方向层面:最少的探测学到对过关有用的规则。可计算版本用 D_t 上的数据拟合代替不可观测的 $d(T^*, T_h)$,再加复杂度项(δ 为 Kronecker delta):

$$L_{fit}(h; D_t) = \sum_i (1 - \delta(R_h(T_h(\hat{z}_i, a_i)), x_{i+1}))$$

$$h_t = \arg \min_{h \in H} C(h) + \lambda L_{fit}(h; D_t)$$

- L_{fit} :已有记录上的预测错误次数;精确回放校验即硬约束 $L_{fit} = 0$ 。
- $C(h)$ 模型复杂度:理想是 Kolmogorov 复杂度 $K(h)$ (不可计算),实际用规范化后的描述长度近似(不是字符数,变量名长不等于复杂);“自由度”只适用于参数模型。
- 形式上与 MDL(最小描述长度)相似,仅作参考。
- 求解是程序空间里的离散搜索(LLM 提出修改,校验器筛选),不是梯度下降。

3.4 玩游戏 cost 与交替优化

$$\min n_{g,\ell} = n_{g,\ell}^{explore} + n_{g,\ell}^{play}$$

- 探索与规划交替,服从同一个全局目标;探索的价值按它让 n^{play} 少了多少衡量。
- 何时值得花代价建模、何时直接行动:Tycho 称为主动抽象(active abstraction)。

3.5 外层:meta-learning 与特权信息

$$\omega^* = \arg \min_{\omega} E_{T \sim p(T)} [L_T(f_{\omega}(D_T^{train}); D_T^{test})]$$

f_{ω} 在任务的 D^{train} 上适应,在 D^{test} 上评估,外层优化 ω 。本题 $\omega = \Lambda$,任务 = 游戏, D^{train} = 前面关卡, D^{test} = 后面关卡:

$$\Lambda^* = \arg \max_{\Lambda} E_{g \sim G_{train}} [S_g(\pi_{f_{\Lambda}})]$$

层	学什么	何时学	比喻
外层 Λ	基础类/交互原语、建模方法、模型权重	离线;若评测顺序运行且规则允许,可在评测中继续积累	身体素质、神经肌肉
内层 h	这个游戏的 Enc_h, T_h, R_h, G_h	评测时现场	具体一套动作

- 特权信息:25 个公开游戏的源码只在训练时可用,外层+内层合起来的程序与真实代码语义一致即学习成功;推理时只剩 L_{fit} 与后续帧预测准确率作代理。这对应 teacher-student 蒸馏,不同于普通 RL(训练与推理信号相同)。
- 与机器人的对比:sim-to-real 部署也看类似 L_{fit} 的一致性,在线适应(如 RMA,凭记忆)只调参数;ARC 要在线改规则结构。
- 与 Lux AI S3 对比:Lux 规则公开,内层只推断参数(学参数);ARC 内层要推断结构(学结构)。

4 方法分类与 A 类:不用 LLM

方法按 h 的形式分三类;离线/联网只是属性。成绩越高越接近 C 类,离线条件下还没有 C 类。

类别	h 的形式	L_fit 校验	方法
A 不用 LLM	q_θ (变不变)、状态图 + 价值模型	无	StochasticGoose、Blind Squirrel、Graph-Based Exploration
B LLM + 文字 h	自然语言规则假设,不可执行	无	Duck、Reki、forge、AERA(LLM 版)、Symbolica
C LLM + 可执行 h	Python 程序	有(回放)	Rodionov EWM、Tycho、OPINE-World

4.1 StochasticGoose(Preview 第 1,12.58%)

内层只学”画面会不会变”, 不学 T_h 与 G_h :

$$q_\theta(x, a) \approx P(\text{画面变化} \mid x, a), \quad \pi(a \mid x) \propto q_\theta(x, a)$$

- q_θ 是在线训练的 CNN:ACTION1-5 一个输出头,点击一个 64×64 输出头——图上位置 (u, v) 的值是”点这个坐标后整张画面会变”的概率,不是这个像素自己变的概率。
- 标签只有变/不变,缓冲区按 (x, a) 哈希去重,最多 20 万条;每进入新关卡清空缓冲区并重置模型,无外层,连跨关卡都没有。
- 目标本身不对:变 $\neq$ 赢;有实验显示在现行计分下即使 q_θ 完美也几乎无收益。

4.2 Blind Squirrel(Preview 第 2,6.71%)

- 状态图:节点为哈希后的帧,边为 (x, a, x') ;剪掉自环、成环动作,并避开已知导致 GAME_OVER 的动作。这是查表式的规则记录,不泛化。
- 价值模型:得分提升后在图上倒推每个状态到赢的最短步数(BFS),以此为标签训练 $V_\psi(x, a)$ 。模型是 ImageNet 预训练的 ResNet-18,自定义输入层与输出头,输入为 64×64 网格加动作 embedding;在同一游戏前几关上微调,第 1 关结束前不用。打分与一个”有效动作模型”结合决定动作。
- 按两层看:外层 = ImageNet 通用视觉特征;内层 = 同一游戏跨关卡的微调。学到的是价值(离赢多近),不是游戏怎么运作;标签对准了目标,设计上优于 4.1,但 Preview 成绩反过来(只有 3 个游戏)。
- 本质:系统探索 + 动态规划生成标签 + 网络拟合,可迭代(expert iteration)。

4.3 Graph-Based Exploration(Preview 第 3)

不训练:把帧分割成组件、按视觉显著性排序动作、维护状态图,优先去最近的未试(状态,动作);并做状态条掩码以减少状态数。在 6 个游戏 52 关中中位数解出 30 关。

5 B 类:LLM + 文字形式的 h

大方向一致:外层 $\Lambda = (\theta, \text{harness})$,内层是上下文学习, h 是一段文字假设;细节差别在动作怎么产生(写程序算 vs 直接选)。

5.1 Duck(Milestone 1 第 1;半私有 1.21%,公开 1.60 ± 0.45)

依据:duck-harness 源码 prompts.py 与配置文件。

- 状态: x_t 及其视图,原始数字网格不给。current_frame 只开放 .ascii(字母编码网格,只许读小区域)、.segmentation、.step、.level、.shape;每轮另附网格图片。.segmentation = $\phi_0(x_t)$:每个节点是 4

邻接同色物体,带 id、color、hash(与位置无关的颜色+形状签名,用于跨帧追踪)、pixels、boundary(外轮廓角点)、children(被包住的物体),另有 adjacency_list。与 1.3 节的物体定义几乎一致。

- **交互**:history、transitions(动作、前后帧、结果)即 D_t ;last_action_result(画面是否变、过关、GAME_OVER);valid_actions 即 A_t ;action([...]) 在 Python 里批量执行真实动作。
- **代码做什么**:帧差分、物体追踪、判断是否只有 HUD 在变、没把握时写有区分度的试探、有把握后写 BFS/寻路搜索最短序列。
- **h 是什么**:对话里的文字”工作 world model”(有哪些物体、动作作用、目标、不确定点、计划)。每次 Python 调用都是全新环境,代码不保留;上下文 32K token,最旧消息被淘汰,h 随之丢失。
- **两层**: θ = 现成 Qwen3.6-27B-FP8,配置里无微调;harness = 系统提示(先验:场景由物体组成、贴边长条多为计时条、不一定有角色;流程:维护 world model、先试探再搜索)+ 工具。 θ 由预训练学到,提示词由人在公开游戏上反复看失败案例修改(人在环路中的离线外层学习)。

$$h_t, a_t = \text{LLM}(\theta, P, D_t)$$

- 官方称 Duck 是 StochasticGoose 的精神续作,指血缘(同为 Tufa Labs,Smit 参与),不是方法继承。

5.2 Reki(Milestone 1 第 2,分数未公布)

视觉 LLM 直接当策略:每轮把最近几帧渲染成带标注的图,交给本地 Gemma-4-31B,要求返回 JSON:变化描述、简短计划、接下来 1-4 个动作;反思记忆约每 10 步刷新。基于官方 GPT-OSS-120B 模板;带 JSON 自修复、合法动作约束、计划队列;手写点击启发式(优先小的、颜色稀少的、像按钮的形状)与”无效签名”(点过从不变的物体类本关不再点);各功能可用环境变量开关做消融。与 Duck 的差别不在多模态(Duck 也用图片),而在不写代码、不搜索、直接选动作。

5.3 forge(Milestone 1 第 3,分数未公布)

流程同 Reki,包装成配置文件控制的框架:多候选动作生成 + 仲裁打分,可选置信度提示(没把握时强制选安全、可撤销的动作);最高分运行关掉了这些机制。作者指出本地公开游戏成绩不能可靠代表 LB。

5.4 AERA(论文 arXiv 2605.25931)

- **LLM 版 agent**:Qwen2.5-0.5B 每步输出 HYPOTHESIS / UNCERTAIN / NEXT_ACTION / REASON;显式三阶段 EXPLORE(预算 $\max(5, \min(30, [0.4 \cdot \text{第一关基线}]))$)→VERIFY(1-3 步证伪)→PLAN(计划/置信度/备选,意外即退回 EXPLORE);用 UNCERTAIN 字段长度代理信念熵做切换,作者自认是弱启发式;记忆只留最近 10 步。公开 25 游戏自报 0.2116(解出 4 个)。
- **私有集 0.30 不是这个 agent**:比赛提交是开局 BFS 预解(每关 180 秒)+ 启发式回退,无学习,更接近 A 类。
- **主要贡献是批评 benchmark**:25 个公开游戏都能用非智能策略到达胜利,另有空坐标漏洞一步绕过 18 个游戏。其 RHAE 不按关卡加权,分数口径与官方不同,不可横比。

5.5 Symbolica Arcgentica(前沿模型,发布首日公开集 36.08)

通过代码与游戏交互的递归 agent 编排;h 隐含,无显式建模 cost。

6 C 类:LLM + 可执行的 h

外层同样是 LLM + harness,区别在于 h 写成可运行的程序并用 L_{fit} 校验;目前全部是前沿 API 模型、公开集、无留出集评测。

6.1 Rodionov EWM(GPT-5.5:58.12,完全解出 15 个;GPT-5.4:41.29,8 个)

- h :固定接口的 Python 代码库(状态表示、转移预测、目标判定、规划器)。
- 建模 cost:每次修改跑校验器,要求 $L_{fit} = 0$ (精确回放);重构降低 $C(h)$,作者称为 MDL 简洁性偏好的近似。
- 在 T_h 里规划,执行时逐步比对,不符即停下修改。每次游玩从干净工作区开始,外层不累积;并审计了非预期信息通道(agent 会主动读游戏源码、人类基线)。
- 后续论文消融:ewma_sv(可执行 + 简化 + 回放校验)vs twma(纯文字 h)。

6.2 Tycho(Opus 5 / GPT-5.6 Sol:100.00;Opus 4.8:88.49,无模型基线 79.07)

- 形式化为参数化、渲染式的确定性 Moore 机;把可行动帧与动画帧、过关帧、失败帧分开。
- h :自由选择状态变量、初始化与转移规则、渲染器、结果分类器的 Python 模拟器;允许单帧不决定下一帧的隐变量(与 3.2 节的 Enc_h, T_h, R_h 同构)。ls20 上生成的 State 包含角色位置、剩余能量、携带的形状颜色、补给、发射器、移动物体方向、迷雾下的地形等。
- 关键变量是何时建、何时用模型:“actor 主动请求 builder”最好;自动修复后模型更准但 RHAE 下降。Opus 5 一次约 \$2.99k。

6.3 OPINE-World(Opus 4.8:78.40,Tycho 文中同模型对比)

把 x_t 解析为物体集合 $s_t = \{o^i\}, o^i = (\kappa^i, \tau^i, v^i)$;配对前后帧物体、算效应签名、填效应表行 (τ^i, a_t, u^i) ;用 Dirichlet 行诊断、类型不确定性与”本体误差”判断抽象是否够用,本体稳定后才合成可执行 T_h ,并以完整下一状态相等校验。

6.4 未归类(h 形式未核实)

- Kepler(Opus 5:100.00): “可审计的 world model”,成本约 \$778,token 比 Tycho 少 36%。
- ARA(AgentNativeResearchLab):边玩边建结构化 h ,保留带置信度的预测账本;分数未查到。

7 在线为什么强:外层 θ 、推理预算与内层表示

现有证据指向:差距主要来自外层模型能力 θ ,其次是推理预算;内层学习机制两边相同,显式 T_h 加校验在前沿模型上只贡献十分左右。

因素	证据	结论
外层 θ (模型权重)	同一 harness 只换模型:EWM GPT-5.5 58.12 vs GPT-5.4 41.29;Tycho Opus 4.8 88.49 vs Opus 5 / GPT-5.6 Sol 100。Duck 用 GPT-5.4 与 EWM 解出的游戏集合相近,成本低一个数量级	能不能解由 θ 决定,成本由 harness 决定
推理预算	Tycho + Opus 5 约 \$2.99k、约 13 亿 token;Kepler 约 8.6 亿 token;Kaggle 单卡 9 小时,Duck 上下文 32K	内层”怎么学”相同,“能学多久”差几个数量级
内层表示(显式 T_h + 校验)	Tycho 无模型基线 79.07,最好的 world model 策略 +9.42	有贡献,但不是主因

- harness 是包在 LLM 外的程序:观察怎么呈现、给哪些工具、提示怎么写、循环与上下文怎么管;在本文框架里是 Λ 中 θ 以外的部分。

- **模型规模**:OpenAI 未公布 GPT-5.4 参数量,但外部估计均在万亿级,比 27B 大约两个数量级;容量不一定也差 100 倍,但是质的区别。
- **混杂因素**:GPT-5.6-sol 等模型发布晚于公开游戏,且未在留出集测试,公开集高分可能部分来自见过这些游戏。

8 背景方法

按作用在内层还是外层归类;标”记忆”的未逐篇现查。

方法	层	学什么	与本题的关系	出处
EMPA(Tsividis et al.)	内层	$\mathcal{H}$ = VGDL 规则空间上的贝叶斯推断 + 探索 + 规划	ARC 效率计分的灵感;VGDL 只能表达简单碰撞类交互	ARC 官方引用
TheoryCoder	内层	LLM 合成代码表达 T_h ,用于双层规划	突破 VGDL 表达力	TMLR 2025
WorldCoder(Tang, Key, Ellis)	内层	写代码 + 交互构建 T_h	程序式 world model 早期工作	2024
Code World Models + MCTS(Dainese et al.)	内层	LLM 生成代码 T_h ,MCTS 引导	同上	2024
PoE-World	内层	T_h = 程序专家的乘积	组合式规则表示	2025
DreamCoder	外层	从多任务学可复用的程序原语库 Λ	“原语库作为先验”最接近的范式	记忆
Voyager	外层	agent 技能库持续积累	技能库跨任务复用	记忆
Dreamer / MuZero	内层	神经网络 T_h + 在想象中规划 / 搜索	需大量交互,不适合首次接触的少样本设定	记忆
MAML / RL ²	外层	ω = 初始化参数 / 会适应的 RNN 权重	meta-learning 标准形式 (3.5 节)	记忆
AlphaGo / AlphaZero	单任务	已知规则下的 SL + RL + MCTS	非 meta-learning;搜索→蒸馏的思路可借用	记忆
RMA	内层(参数)	在线估计环境参数	只调参数不改结构,与 ARC 对照	记忆

9 对照总表

按类别排列;联网成绩是公开集 RHAE(百分制),离线、Preview 与 AERA 自报口径不同,不可直接横比。

方法	类	条件	成绩	外层 Λ	内层 h	建模 cost	玩游戏 cost
Tycho	C	联网,Opus 5 / GPT-5.6 Sol	公开 100.00	固定框架 + LLM	可执行模拟器 (含隐变量)	L_{fit} 校验	模型内规划;建/用模型由 actor 决定
Tycho	C	联网,Opus 4.8	公开 88.49	同上	同上	同上	同上
OPINE-World	C	联网,Opus 4.8	公开 78.40	物体解析 + 效应表	本体稳定后合成 T_h	本体误差;下一状态相等	未读
Rodionov EWM	C	联网,GPT-5.5	公开 58.12	固定接口 + 校验器 + 重构	Python 代码库	$L_{fit} = 0$ + 降低 $C(h)$	模型内规划,执行比对

方法	类	条件	成绩	外层 Λ	内层 h	建模 cost	玩游戏 cost
Symbolica	B	联网,前沿模型	公开 36.08	LLM + harness	隐含	无显式	无显式
StochasticGoose	A	Preview	12.58%	无	$q_\theta(x, a)$, 每关重置	变/不变交叉熵	$\pi \propto q_\theta$
Blind Squirrel	A	Preview	6.71%	ImageNet ResNet-18	状态图 + V_ψ , 跨关卡微调	无(精确记录)	价值回归
Graph-Based Exploration	A	Preview	第 3, 中位数 30/52 关	无	状态图	无	最近未试动作
Duck	B	Kaggle 离线	半私有 1.21%	Qwen3.6-27B + 提示词 + 工具	对话里的文字 world model	无显式	LLM 写 BFS 等搜索
Reki	B	Kaggle 离线	未公布	Gemma-4-31B + 点击先验	反思记忆 + 失效集合	无显式	LLM 直接选 1-4 步
forge	B	Kaggle 离线	未公布	同 Reki	同 Reki	无显式	LLM 直接选
AERA(LLM 版)	B	论文	公开 0.2116(自报口径)	Qwen2.5-0.5B + 三阶段流程	HYPOTHESIS 字段	UNCERTAIN 长度代理熵	PLAN 阶段输出计划
Kepler	未归类	联网, Opus 5	公开 100.00	可审计 WM 框架 + LLM	未读	未读	未读
ARA	未归类	联网, Opus 4.8 等	未查到	LLM	结构化 h	预测账本	未读

10 空白与开放问题

三处空白:显式外层 Λ (离线学原语库,评测中可更新);离线本地模型条件下的 C 类方法;“怎么学”(探索选哪个动作、假设如何取舍)的显式化。

- Milestone 1 第 2、3 名分数;Milestone 2(9/30 截止)结果
- Kepler、ARA 的 h 形式与 cost 定义
- Kaggle 是否同进程顺序跑 110 个游戏;规则是否允许评测中跨游戏累积 Λ
- 规则是否允许用 API 模型生成的数据训练本地模型
- Kaggle 服务端计分是否与 SDK scorecard.py 一致(RESET 计入规则)
- 可信判据中 ε 的取值;判据是否闭合、是否需要证明
- $C(h)$ 的具体度量(规范化描述长度的定义)

11 推演记录:审阅中的关键修正

这一节记录从第一版到本版的来回:哪里不闭合、为什么改、改成什么。

#	第一版的写法	审阅中发现的问题	本版
1	ϕ 把帧变成”物体级状态”	物体、状态未定义;物体会递归包含、互相遮挡	s 为树;最小外接矩形左上角 + 掩码 + 颜色图 + 层次;遮挡用历史补全; ϕ_o 是先验、 ϕ_h 属于假设
2	用质心表示位置	质心不能把物体画回帧上	位置 = 帧坐标下最小轴对齐矩形左上角

#	第一版的写法	审阅中发现的问题	本版
3	L_fit 直接在 x 上算 T_h(x, a)	T* 不可观测的根源是隐变量 z; T_h 应作用在 z 上	Enc_h 用历史估计 $\hat{z}$, T_h($\hat{z}$, a), R_h 渲染回帧后比较
4	L_len 模型长度; “得到 MDL 形式”	长不等于复杂(变量名长); 是自己建模不是套 MDL	C(h) 模型复杂度(理想 K(h)); “形式上与 MDL 相似”
5	“h 足够可信”	未定义	候选假设在计划路径上预测一致; 与探索判据同源; ϵ 待定
6	RESET 是否计入未核实	应查源码	普通 RESET 计 1 动作; 完全重置新开记录
7	第 4/5 章按离线/联网分	B 类离线方法同样是 LLM + 提示词, 也在学 T_h(文字形式); 掩盖了真正的区别	按 h 的形式分 A/B/C 三类
8	Duck 外层” 没有学习”	外层有模型, 学习方法是人给思路 and 提示	$\Lambda = (\theta, P, \text{工具})$; θ 由预训练学, P 由人在环路中学
9	Duck 的 h 以代码保存	源码: Python 每次全新环境	h 是对话里的文字, 随上下文淘汰
10	Duck 是 StochasticGoose 的方法续作	两者方法完全不同	只是血缘(同一实验室)
11	Blind Squirrel 的 ResNet 从头训练、只限同关	作者仓库: ImageNet 预训练, 跨关卡微调	外层视觉特征 + 内层跨关卡泛化
12	点击概率图” 每个像素变的概率”	描述不清	点该坐标后整张画面会变的概率
13	AERA 0.30 归在 LLM agent 名下	原文: 比赛提交是 BFS 预解	分开描述, 标注口径不同
14	在线强于离线原因未分解	是外层、内层还是推理?	主因 θ , 其次推理预算, 内层表示约 +9 分
15	“GPT-5.4 不一定更大”	过于保守	外部估计均为万亿级, 约大两个数量级

其他讨论要点: 普通 RL 训练与推理信号相同, 公开游戏源码是特权信息(teacher-student); 外层 = “有哪些基础类和交互”, 内层 = “这个游戏怎么定义这些类、怎么赢”; 不以刷榜为目标, 方法好自然 paper 好。

Sources

- [ARC Prize 2026 Milestone 1 官方博客](#)
- [ARC-AGI-3 Preview 30-day learnings](#)
- [Duck harness 源码](#) · [Tufa Labs write-up](#)
- [StochasticGoose 源码](#)
- [Blind Squirrel 源码](#)
- [Graph-Based Exploration \(arXiv 2512.24156\)](#)
- [AERA \(arXiv 2605.25931\)](#)
- [Executable World Models \(arXiv 2605.05138\)](#) · [代码与运行产物](#)
- [Do Coding Agents Need Executable World Models... \(arXiv 2607.15439\)](#)
- [Tycho \(arXiv 2607.28287\)](#) · [代码](#)
- [Kepler \(arXiv 2610.00834\)](#)
- [OPINE-World \(arXiv 2607.01531\)](#)
- [ARA 轨迹示例](#)

- [ARC-AGI-3 技术报告](#)
- [arc-agi 0.9.9 / arcengine 源码\(PyPI,scorecard.py、models.py\)](#)